

StonkBench: Unified Benchmark for Synthetic Market Generation for Historical Financial Time Series

Albert Ho*
uyenlam.ho@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Eddison Pham*
eddison.pham@mail.utoronto.ca
University of Toronto
Toronto, Ontario, Canada

Abstract

Synthetic data generation has become increasingly important in financial applications, particularly with generating market data such as financial securities prices. With advancements in generative architectures in text, image, and audio domains, practitioners and researchers in financial markets have shown growing interest in applying these techniques to generate market data like time series of equity prices for research and robust trading strategies backtesting. However, the current literature in this domain lacks a unified benchmark for evaluating the quality of synthetic market data for fair and reliable comparisons across different families of models. To address this gap, we introduce StonkBench, a unified benchmark for synthetic multivariate time series generation of daily equity returns through a comprehensive and standardized evaluation framework that balances statistical properties, domain-specific stylized facts observed in market data and practical downstream tasks practitioners care about, such as portfolio hedging, option price delta hedging, and alpha generation.

Keywords

Synthetic Data, Market Generator, Financial Time Series, Benchmarking

ACM Reference Format:

Albert Ho and Eddison Pham. 2025. StonkBench: Unified Benchmark for Synthetic Market Generation for Historical Financial Time Series. In . ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Synthetic market data generation has become an important tool in finance, particularly for time series such as asset returns, trading volume, and limit order books [12, 18]. It can help researchers simulate fat-tail behavior, volatility clustering, and back-testing scenarios for prediction and hedging algorithms [20].

Despite this growth, evaluation practices remain poorly standardized. Stenger et al. surveyed 56 studies and found little consensus on the metrics used or how they should be interpreted [20]. As a result, comparisons across methods are often difficult and their relative strengths remain unclear.

1.1 A Brief Overview on Market Generators

Synthetic market data are multivariate time series related to financial assets and market microstructures, such as limit order books, trading volume, volatility surfaces, and asset returns [3, 12, 20].

*Both authors contributed equally to this research.

Researchers and practitioners are mainly interested in them for two reasons [12]: forecasting and prediction, where synthetic paths can support model training and evaluation, and simulating market scenarios, where they can reproduce rare events and stress-test strategies. Recent generative models such as GANs and VAEs, especially conditional variants, are attractive because they can learn conditional distributions of financial time series rather than only point estimators [12].

1.2 Limitations in Market Generator Literature

Existing evaluation practices for synthetic time series remain fragmented, with no broadly accepted standard [20, 22]. Many measures are poorly defined, inconsistently implemented, and hard to reproduce [20]. This is especially problematic for generative tasks, where evaluation must capture more than marginal similarity and instead reflect temporal structure, diversity, and downstream utility [12, 20]. Our review highlights four main limitations:

- [L1] **Inconsistent evaluation metrics:** Different studies use different measures, making comparisons difficult [20].
- [L2] **Lack of standardized datasets and preprocessing:** Benchmarking is often done on different assets, horizons, and preprocessing choices, which weakens comparability [20].
- [L3] **Narrow evaluation scope:** Many studies focus on marginal distributions and overlook temporal dependencies, diversity, and downstream usefulness [12, 20].
- [L4] **Reproducibility issues:** Missing implementations and detailed protocols make results hard to verify across studies [18, 20].

1.3 Our Contributions

We develop a unified benchmark for synthetic equity-return generation that combines statistical and utility-based evaluation. Building on the taxonomy of Stenger et al. [20], the framework of Ang et al. [1], and utility-focused work by Boursin et al. and Fecamp et al. [4, 7], we study how different metrics trade off in practice. Our key contributions are:

- [C1] **A consolidated evaluation framework:** We review and integrate measures from prior work to provide a more systematic benchmark for market generators [1, 4, 20].
- [C2] **Unified statistical and utility measures:** We combine fidelity metrics with downstream utility metrics in a single evaluation suite.
- [C3] **An open benchmark framework:** We provide an open-source setup to support more consistent comparisons in future market-generator research.

1.4 Research Questions

Our benchmark addresses three main questions:

- [Q1] How do different training sequence lengths affect generator performance?
- [Q2] Are there trade-offs between fidelity, diversity, and downstream utility?
- [Q3] Do modern deep-learning generators outperform classical parametric models in practical tasks?

These questions are explored further in §4 and the evaluation sections that follow.

2 Preliminaries

2.1 Problem Definition

Let us formally define the market generation problem. Given a financial time series of asset price movements $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_I)^T = (\mathbf{X}_t)_{t=1}^T \in \mathbb{R}^{T \times I}$ with I individual securities spanning time T , let $p(\mathbf{X})$ denote its real distribution. The objective of the market generator is to synthesize a time series $\hat{\mathbf{X}} = (\hat{\mathbf{X}}_t)_{t=1}^T$ such that its distribution $q(\hat{\mathbf{X}})$ approximates $p(\mathbf{X})$ while preserving key stylized facts, generating diverse samples, and improving downstream task training and evaluation. More importantly we wish to evaluate the generalization capabilities of $q(\hat{\mathbf{X}})$ against unseen data $p(\mathbf{X}_{\text{new}})$.

2.2 Scope of Project

2.2.1 Evaluated Methods. Our benchmark covers models from classical parametric methods to modern deep-learning architectures. We selected models that meet one of three criteria: (1) proven industry adoption, (2) research community acceptance, and (3) recent methodological innovation.

Classical parametric models rely on predefined statistical distributions and assumptions about the data-generating process. They remain common in financial institutions because of their interpretability and strong theoretical foundations.

We also evaluate modern deep-learning approaches, especially models that learn the data distribution directly from observations rather than assuming a specific form. These include generative adversarial networks and variational autoencoders, which represent the current state of the art in synthetic market data generation.

An overview of the market-generator classes is provided in §3.

2.2.2 Datasets. The benchmark aims to generate realistic and diverse synthetic time series from daily financial asset prices over 15 years (2010–2025), collected from Yahoo Finance. To evaluate cross-asset behavior, we include 20 US large-cap stocks from several sectors and 5 popular US index-tracking ETFs (see table 1)

2.2.3 Scope of Evaluation Taxonomical Criteria. Our evaluation framework follows the taxonomy proposed by Stenger et al. [20] and incorporates measures from multiple papers in the field.

We integrate evaluation metrics from several sources while maintaining a structured taxonomy, providing broad coverage of the key aspects of evaluation and setting a standard for future market-generator research. A full description of the evaluation suite is provided in §5.1 and §6.

Table 1: Asset classification used in StonkBench.

Ticker	Name	Type	Sector/Exposure
AAPL	Apple	Stock	Technology
MSFT	Microsoft	Stock	Technology
NVDA	NVIDIA	Stock	Technology
AVGO	Broadcom	Stock	Technology
JPM	JPMorgan Chase	Stock	Financials
LLY	Eli Lilly	Stock	Healthcare
UNH	UnitedHealth Group	Stock	Healthcare
AMZN	Amazon	Stock	Consumer Disc.
TSLA	Tesla	Stock	Consumer Disc.
CAT	Caterpillar	Stock	Industrials
UNP	Union Pacific	Stock	Industrials
META	Meta Platforms	Stock	Communication
NFLX	Netflix	Stock	Communication
PG	Procter & Gamble	Stock	Consumer Staples
COST	Costco Wholesale	Stock	Consumer Staples
XOM	Exxon Mobil	Stock	Energy
CVX	Chevron	Stock	Energy
NEE	NextEra Energy	Stock	Utilities
PLD	Prologis	Stock	Real Estate
LIN	Linde	Stock	Materials
SPY	S&P 500	ETF	S&P 500
QQQ	Nasdaq-100	ETF	Nasdaq-100
XLF	Finance	ETF	U.S. Financials
IWM	Russell 2000	ETF	Small Caps
XLV	Health Care	ETF	U.S. Healthcare

3 Overview of Market Generator Methods

We categorize synthetic financial time series generation methods into two main families: classical stochastic (parametric) models, which rely on explicit assumptions about the data-generating process, and modern deep learning models, which learn complex dependencies directly from data without predefined functional forms. The following subsections provide a detailed overview of the methods included in our benchmark.

Throughout the paper, we denote by $\mathbf{S}_t$ and $\mathbf{R}_t$ the vectors of raw prices and log returns of the I assets at time t , with scalar entries $S_{t,i}$ and $R_{t,i}$ for asset i , and by lowercase r_t the simple (non-log) return. $\mathbf{X}$ denotes a generic multivariate series, interchangeable between $\mathbf{S}$ and $\mathbf{R}$; bold symbols denote vectors or matrices, while non-bold symbols denote scalars.

3.1 Classical Stochastic (Parametric) Methods

Parametric models assume a specific functional form for the data generating process, characterised by a finite set of parameters. These models are interpretable and analytically tractable, allowing for closed-form solutions in many cases. However, they may struggle to capture complex stylized facts or high-dimensional dependencies beyond their structural assumptions. Below we outline several widely-used parametric models in financial markets.

- [A1] **Geometric Brownian Motion (GBM).** The price process follows a continuous-time stochastic process with constant

drift vector μ and volatility $\Sigma^{1/2}$ ¹.

$$dS_t = \mu S_t dt + \Sigma^{1/2} S_t dB_t. \quad (1)$$

[A2] **Ornstein-Uhlenbeck (OU)**. A mean-reverting Gaussian process for log-prices or spreads. Model-specific parameters are $\theta > 0$ (reversion rate), μ (long-term mean), and σ (volatility).

$$dS_t = \theta(\mu - S_t) dt + \Sigma^{1/2} dB_t. \quad (2)$$

[A3] **Merton Jump Diffusion (MJD)**. Extends the GBM equation (1) by incorporating normally-distributed jumps. Model-specific parameters are λ (jump intensity), μ_j and σ_j (mean and standard deviation of jump sizes).

$$\frac{d(\log S_t)}{\log S_t} = \mu dt + \sigma dB_t + J dN_t, \quad (3)$$

where $J \sim \mathcal{N}(\mu_j, \sigma_j^2)$, $N_t \sim \text{Poisson}(\lambda t)$.

[A4] **Double-Exponential Jump Diffusion (DEJD)**. Another jump-diffusion model with asymmetric double-exponential jump sizes modifying equation (1). Model-specific parameters are p (probability of upward jump) and η_1, η_2 (exponential parameters for upward/downward jumps).

$$\frac{d(\log S_t)}{\log S_t} = \mu dt + \sigma dB_t + Y dN_t, \quad (4)$$

$$Y = \begin{cases} \text{Exp}(\eta_1), & \text{with probability } p, \\ -\text{Exp}(\eta_2), & \text{with probability } 1 - p. \end{cases}$$

[A5] **GARCH(1,1)**. A discrete-time conditional heteroskedastic model for simple returns. Model-specific parameters are $\omega > 0$, $\alpha \geq 0$, $\beta \geq 0$ (GARCH coefficients) and $\mathcal{L}$ (innovation law, usually standard Normal or Student-t). Here r_t denotes simple, non-log returns. Captures volatility clustering via time-varying conditional variance.

$$\begin{aligned} dr_t &= \sigma_t dz_t, \quad z_t \sim \mathcal{L}, \\ \sigma_t^2 &= \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2. \end{aligned} \quad (5)$$

3.2 Deep Learning Methods

Deep learning methods learn complex patterns directly from data without assuming a fixed parametric form. They are flexible enough to model nonlinear and high-dimensional financial dependencies, but they typically require more data, tuning, and computation than classical approaches. In this paper, we focus on three major deep generative families: Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and diffusion models.

3.2.1 Generative Adversarial Networks (GANs). GANs model generation as a two-player game between a generator and a discriminator. The generator tries to create realistic samples, while the discriminator learns to distinguish real from synthetic data [9]. In this benchmark, we include three GAN variants tailored to sequential and financial data.

[A6] **QuantGAN**. A GAN tailored for financial time series that operates on log-returns mapped onto a grid of quantile bins and parameterizes both the generator and the discriminator with temporal convolutional networks (TCNs) [23]. The generator G and discriminator D are trained with the Wasserstein GAN with gradient penalty (WGAN-GP) objective [2]:

$$\begin{aligned} \min_G \max_D \mathbb{E}_{\mathbf{R} \sim \mathbb{P}}[D(\mathbf{R})] - \mathbb{E}_{\hat{\mathbf{R}} \sim \mathbb{Q}}[D(\hat{\mathbf{R}})] \\ + \lambda_{\text{GP}} \mathbb{E}_{\hat{\mathbf{R}}} \left[\left(\|\nabla_{\hat{\mathbf{R}}} D(\hat{\mathbf{R}})\|_2 - 1 \right)^2 \right], \end{aligned} \quad (6)$$

where $\mathbb{P}$ and $\mathbb{Q}$ denote the real and generated distributions of the log-return series $\mathbf{R}$, and $\hat{\mathbf{R}}$ are points sampled along straight lines between real and generated windows. By matching the full return distribution, QuantGAN aims to reproduce tail behaviour and risk-sensitive statistics (e.g., VaR/CVaR) important for downstream financial tasks.

[A7] **Conditional Sig-Wasserstein GAN (SigCWGAN)**. SigCWGAN replaces the neural discriminator with a linear functional on the path signature $\text{Sig}(\mathbf{X})$ of the time series, giving rise to the conditional Sig-Wasserstein distance [16]:

$$\begin{aligned} \text{SW}_1(\mathbb{P}, \mathbb{Q}) = \sup_{\ell: \|\ell\|_* \leq 1} \mathbb{E}_c \left[\left| \mathbb{E}_{\mathbf{X} \sim \mathbb{P}(\cdot|c)}[\ell(\text{Sig}(\mathbf{X}))] \right. \right. \\ \left. \left. - \mathbb{E}_{\hat{\mathbf{X}} \sim \mathbb{Q}(\cdot|c)}[\ell(\text{Sig}(\hat{\mathbf{X}}))] \right| \right], \end{aligned} \quad (7)$$

where $\text{Sig}(\cdot)$ denotes the signature transform of a path, ℓ ranges over linear functionals on the signature space, and c is a conditioning variable (e.g. volatility). The generator is trained to minimize this distance, which yields more stable training than classical GAN critics while targeting conditional return distributions.

[A8] **PCF-GAN**. PCF-GAN compares path-space measures through their characteristic functions (PCF distance) [15]:

$$\begin{aligned} \text{PCF}(\mathbb{P}, \mathbb{Q}) &= \sup_{\zeta \in \mathcal{Z}} |\phi_{\mathbb{P}}(\zeta) - \phi_{\mathbb{Q}}(\zeta)|, \\ \phi_{\mathbb{P}}(\zeta) &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}}[e^{i\langle \zeta, \mathbf{X} \rangle}], \end{aligned} \quad (8)$$

where $\phi_{\mathbb{P}}$ is the characteristic function of the path-valued random variable $\mathbf{X}$ and ζ a frequency parameter. The generator is trained to minimize this distance, which is injective on the path space and can be estimated without a learned discriminator, providing a statistically grounded divergence for sequential data.

3.2.2 Variational Autoencoders (VAEs). VAEs are latent-variable generative models that learn a compact representation of the data and reconstruct realistic samples from it. Recurrent variants impose temporal structure on the latent space so that generated trajectories remain coherent across time.

[A9] **Variational Recurrent Neural Network (VRNN)**. VRNN embeds a VAE at every time step of an RNN, with the prior over the latent variable z_t conditioned on the previous hidden state h_{t-1} [5]:

$$\begin{aligned} z_t &\sim \mathcal{N}(m_{0,t}, \text{diag}(s_{0,t}^2)), \quad [m_{0,t}, s_{0,t}] = \psi^{\text{prior}}(h_{t-1}), \\ h_t &= f_{\psi}(h_{t-1}, \mathbf{X}_t, z_t), \\ \mathbf{X}_t &\sim p_{\psi}(\mathbf{X}_t | z_t, h_{t-1}), \end{aligned} \quad (9)$$

¹In the following section, the volatility matrix $\Sigma^{1/2}$ is the Cholesky decomposition of the sample variance matrix Σ of a given data

trained by maximizing the ELBO with the KL term regularizing the latent space, allowing the model to capture stochastic dependencies in sequential data.

- [A10] **Kalman-VAE**. Kalman-VAE is a VAE whose latent dynamics follow a linear Gaussian state-space model, so that posterior inference is performed exactly with a Kalman filter/smoothener [8]:

$$\begin{aligned} z_t &= \tanh(Az_{t-1} + \epsilon_t), \quad \epsilon_t \sim \mathcal{N}(0, Q), \\ X_t &= \tanh(Cz_t + \delta_t), \quad \delta_t \sim \mathcal{N}(0, V), \end{aligned} \quad (10)$$

where A and C are (possibly time-varying) transition and emission matrices, and Q and V are the process and observation noise covariances, respectively. Exact Gaussian inference in the latent space yields a tractable KL divergence and disentangles the latent dynamics from the observation model.

3.2.3 Diffusion Models. Diffusion (or score-based) generative models corrupt data through a forward noise-adding process and learn a reverse denoising process that generates samples from noise [11]. Given a noise schedule $\beta_1, \dots, \beta_K$, the forward process and the reparameterized noised samples are

$$\begin{aligned} q(\mathbf{X}^k | \mathbf{X}^{k-1}) &= \mathcal{N}(\sqrt{1 - \beta_k} \mathbf{X}^{k-1}, \beta_k \mathbf{I}), \\ \mathbf{X}^k &= \sqrt{\alpha_k} \mathbf{X}^0 + \sqrt{1 - \alpha_k} \epsilon, \quad \alpha_k = \prod_{i=1}^k (1 - \beta_i), \end{aligned} \quad (11)$$

and training minimizes the simplified denoising score-matching objective

$$\mathcal{L} = \mathbb{E}_{k, \mathbf{X}^0, \epsilon} \left[\|\epsilon - \epsilon_\psi(\mathbf{X}^k, k)\|^2 \right], \quad (12)$$

- [A11] **TimeGrad**. TimeGrad is an autoregressive diffusion model for multivariate probabilistic time series [19]. At each time step t , a denoising network conditioned on the hidden state h_t of a recurrent encoder models the conditional distribution of $\mathbf{X}_t$ given the history $\mathbf{X}_{<t}$:

$$\mathcal{L}_t = \mathbb{E}_{k, \mathbf{X}_t^0, \epsilon} \left[\|\epsilon - \epsilon_\psi(\mathbf{X}_t^k, k, h_t)\|^2 \right], \quad (13)$$

enabling sample-based forecasts of the full multivariate return distribution.

- [A12] **Diffusion-TS (TSDiff)**. Diffusion-TS combines an interpretable encoder that decomposes the input series into trend and seasonal components with a conditional diffusion decoder [24]. The encoder maps the input to a latent code $\mathbf{c} = f_\psi(\mathbf{X}^0)$ capturing these interpretable components, and the decoder generates samples conditioned on $\mathbf{c}$:

$$\mathcal{L} = \mathbb{E}_{k, \mathbf{X}^0, \epsilon} \left[\|\epsilon - \epsilon_\psi(\mathbf{X}^k, k, \mathbf{c})\|^2 \right], \quad (14)$$

where the decomposition into interpretable basis functions (e.g. Fourier and linear trends) is applied before diffusion, exposing the trend-seasonal structure of the generated series.

3.3 Block Bootstrap Methods

Non-parametric methods do not require a fixed parametric distributional form. Block bootstrap approaches resample contiguous blocks of a time series to preserve the short-range dependence structure of the observed data while generating new series; below,

X generically denotes the series, which in our benchmark is the log-return series $\mathbf{R}$.

- [A13] **Moving Block Bootstrap (MBB)**. MBB resamples blocks of fixed length l uniformly with replacement from the observed series and concatenates them [13]. For a series $X_1, \dots, X_n$, overlapping blocks $B_i = (X_i, \dots, X_{i+l-1})$, $i = 1, \dots, N$ with $N = n - l + 1$, are formed; drawing $k = \lceil n/l \rceil$ blocks i.i.d. and concatenating them yields a bootstrap sample

$$\begin{aligned} X^* &= (B_{I_1}, \dots, B_{I_k}), \\ I_j &\stackrel{\text{iid}}{\sim} \text{Unif}\{1, \dots, N\}, \end{aligned} \quad (15)$$

truncated to length n . This preserves within-block dependence at the cost of a discontinuity at block boundaries.

- [A14] **Stationary Block Bootstrap (SBB)**. SBB draws block lengths from a geometric distribution, so the resulting resampled series is (strictly) stationary [17]. Each block starts at a uniformly chosen index and has random length $l \sim \text{Geom}(\varrho)$ with expected block length $1/\varrho$; blocks are wrapped around the circular series and concatenated until the desired length n is reached:

$$\begin{aligned} X^* &= (B_{I_1}^l, \dots, B_{I_k}^l), \\ B_i^l &= (X_i, \dots, X_{i+l-1}), \\ I_j &\sim \text{Unif}\{1, \dots, n\}, \quad l_j \sim \text{Geom}(\varrho), \end{aligned} \quad (16)$$

where indices in each block are taken modulo n (wrapping around the end of the series). Random block lengths avoid the periodicity artifacts of the fixed-length MBB while maintaining dependence structure.

4 Market Generator Framework

It is important to have standardized generation framework as part of our efforts to create a unified benchmark [C3]; this section outlines the details of the dataset used, preprocessing, and generation framework used by StonkBench for models to train on historical market data.

4.1 Data preprocessing

4.1.1 Dataset Split. Given our stock price dataset, we perform a chronological ten-two-three year partition into training, validation, and testing datasets from 2010 to 2025, where $D = D^{\text{train}}, D^{\text{val}}, D^{\text{test}}$ denotes the concatenation. This deviates from the training scheme proposed by TSGBench [1]; we wish to hold out our testing time-horizon from the training set completely to evaluate model generalization capabilities.

4.1.2 Log Return Transformation. We then transform the price time series for both sets into a log return time series, which is a common practice in financial modeling to stabilize variance and make the data more suitable for analysis. The return $\mathbf{R}_t$ at time t is calculated as ^{2 3}:

$$\mathbf{R}_t := \log \left(\frac{S_t}{S_{t-1}} \right) = \log(S_t) - \log(S_{t-1}), \quad (17)$$

²The log return operation is applied channel-wise to the time series.

³From now, on we refer to D as the log return time series dataset and notate it as $\mathbf{R}_t$ to avoid notation cluttering, unless explicitly stated otherwise.

4.1.3 Sliding Window Transformation. The deep learning models considered in §3.2 require a fixed-length data for training and generation. Therefore, to standardize input and data generation, while preserving temporal dependencies, we apply a sliding window transformation to the log return time series. Given a window length $L \in \mathbb{N}$, we applying a sliding window operation W_L over the time series as follows.

$$W_L : (\mathbf{R}_t)_{t=1}^T \mapsto \{\mathbf{w}_t := (\mathbf{R}_t, \mathbf{R}_{t+1}, \dots, \mathbf{R}_{t+L}) : 1 \leq t \leq T - L\} \quad (18)$$

We denote the resulting set of windows, for a given data partition from D , as $\mathcal{D}^i := W_L(D^i)$ for $i \in \{\text{train}, \text{val}, \text{test}\}$. For the purpose of research, we experiment our generation training with window lengths $L \in \{21, 63, 126, 252\}$, representing general the number of trading days in a month, a quarter, half a year and a year, respectively⁴.

4.2 Market Generator Training and Generation

We train each market generator on the log-return training data to produce synthetic windows, each with shape $L \times I$. Formally, for each generator G_g , $g \in \mathcal{G}$, we generate N synthetic windows and obtain the generated set $\hat{\mathcal{D}}_g = \{\hat{\mathbf{w}}_n : \hat{\mathbf{w}}_n \sim G_g(\mathbf{w} \mid D^{\text{train}})\}_{n=1}^N$. Here, $\mathcal{G}$ is the index set of market generators considered in §3, $\hat{\mathbf{w}}_n$ denotes a synthetic window of size $L \times I$, and N is the number of synthetic windows generated for each model, which we set to $N = 1000$.

4.2.1 Non-deep Learning Model Generation. Parametric and bootstrap generators are fit on D^{train} and then sample windows directly from the learned process or resampled data.

$$\hat{\mathbf{S}}_{n_l} \sim G_g(\mathbf{S} \mid D^{\text{train}}), \quad \hat{\mathbf{w}}_n := (\hat{\mathbf{S}}_{n_1}, \dots, \hat{\mathbf{S}}_{n_L}), \quad \hat{\mathcal{D}}_g = \{\hat{\mathbf{w}}_n\}_{n=1}^N. \quad (19)$$

4.2.2 Deep Learning Model Generation. Deep-learning frameworks in this paper (GAN, VAE, DDPM, and signature-based models) are trained on the sliding-window transform $W_L(D^{\text{train}})$ (see §4.1). They then generate N independent windows of size L , each with shape $L \times I$, directly from the learned window distribution:

$$\hat{\mathbf{w}}_n \sim G_g(\mathbf{w} \mid W_L D^{\text{train}}), \quad \hat{\mathcal{D}}_g = \{\hat{\mathbf{w}}_n\}_{n=1}^N. \quad (20)$$

5 Statistical Evaluation Measures

The statistical metrics considered in this paper are used to evaluate either the realism of synthetic market data to its trained history or generative novelty in market scenarios. The following subsections describes the metrics considered and the evaluation framework applied.

5.1 Statistical Measures

As described in [L1], our contribution [C1] is based on a well-established taxonomy framework [20] while introducing our industry-specific stylized facts to holistically evaluate the characteristics of synthetic market data. We divide our statistical metrics

⁴TSGBench determined the window length for deep learning training and generation depending on the dataset by finding the maximum auto-correlation of that time series dataset [1]

5.1.1 Fidelity Measures. These metrics quantify how well synthetic data captures key marginal and second-order properties of real data. We employ the following feature-based metrics:

[M1] Marginal Distribution Difference (MDD): To measure the distance between empirical marginal distributions of real versus synthetic data, we use the 1-Wasserstein distance (a.k.a. Earth Mover's distance). This was chosen over the popular Kullback-Leibler divergence or the derivatives Jensen-Shannon divergence due to its better numerical stability and ability to capture distributional differences even when supports do not overlap [2].

$$\text{MDD} = W_1(\hat{F}_{\text{real}}, \hat{F}_{\text{synth}}),$$

$$W_1(F, G) = \int_0^1 |F^{-1}(u) - G^{-1}(u)| du,$$

where $\hat{F}_{\text{real}}$ and $\hat{F}_{\text{synth}}$ are empirical CDFs.

The following four metrics capture differences in the first four statistical (central) moments between real and synthetic data.

[M2] Mean Difference (MD):

$$\text{MD} = |\mu_{\text{real}} - \mu_{\text{synth}}|.$$

[M3] Standard Deviation Difference (SDD):

$$\text{SDD} = |\sigma_{\text{real}} - \sigma_{\text{synth}}|.$$

[M4] Skewness Difference (SD):

$$\text{SD} = |\gamma(\mathbf{R}_{\text{real}}) - \gamma(\mathbf{R}_{\text{synth}})|,$$

$$\gamma(\mathbf{R}) = \frac{\mathbb{E}[(\mathbf{R} - \mu)^3]}{(\sigma)^3}.$$

[M5] Kurtosis Difference (KD):

$$\text{KD} = |\kappa^{\text{ex}}(\mathbf{R}_{\text{real}}) - \kappa^{\text{ex}}(\mathbf{R}_{\text{synth}})|,$$

$$\kappa^{\text{ex}}(\mathbf{R}) = \frac{\mathbb{E}[(\mathbf{R} - \mu)^4]}{(\sigma)^4} - 3.$$

5.1.2 Visualization Methods. Visual tools complement numeric metrics for face-validity and communication [1].

[M6] t-SNE: 2D embeddings of window-level features for real versus synthetic overlap and cluster structure.

[M7] Distribution: Histograms of returns to visualize empirical distribution.

5.1.3 Diversity Measures. We are also interested if the SDGFTS can generate data not observed but follow the same distribution as the real data. Therefore, we include the following intra-class distance metrics. The inclusion of these metrics are chose to ensure that mode collapse is prevented.

[M8] Intra-Class Euclidean Distance (ICD-ED):

$$\text{ICD-ED} = \frac{2}{R(R-1)} \sum_{1 \leq a < b \leq R} \|\mathbf{R}_a - \mathbf{R}_b\|_2,$$

where $\mathbf{R}_a \in \mathbb{R}^L$ is the a -th sample.

[M9] Intra-Class Dynamic Time Warping (ICD-DTW):

$$\text{ICD-DTW} = \frac{2}{R(R-1)} \sum_{1 \leq a < b \leq R} d_{\text{DTW}}(\mathbf{R}_a, \mathbf{R}_b).$$

5.1.4 Efficiency Measures. Generation efficiency is crucial for practical applications, especially in high-frequency trading or real-time risk management, where rapid data synthesis is required. Thus, we include the following efficiency metric [20].

[M10] Generation Time: Wall-clock time to generate S samples:

$$T_{\text{gen}} = t_1 - t_0 \quad (\text{seconds for } 1,000 \text{ samples}).$$

5.1.5 Stylized Facts Measures. Canonical stylized facts:

[M11] Autocorrelation Difference (ACD): Absolute difference in lag-1 autocorrelation:

$$\rho(1) = \frac{\sum_{t=2}^L (\mathbf{R}_t - \bar{\mathbf{R}})(\mathbf{R}_{t-1} - \bar{\mathbf{R}})}{\sum_{t=1}^L (\mathbf{R}_t - \bar{\mathbf{R}})^2},$$

$$\text{ACD} = |\rho_{\text{real}}(1) - \rho_{\text{synth}}(1)|.$$

[M12] Volatility Clustering (VC): Lag-1 autocorrelation of squared or absolute returns measures volatility clustering, a stylized fact where large price movements tend to cluster in time [6]:

$$\rho_{\mathbf{R}^2}(1) = \text{Corr}((\mathbf{R}_t)^2, (\mathbf{R}_{t-1})^2),$$

$$\rho_{|\mathbf{R}|}(1) = \text{Corr}(|\mathbf{R}_t|, |\mathbf{R}_{t-1}|).$$

[M13] Long Memory / Slow Decay (LMSD): To quantify long-range dependence in volatility, we follow Cont (2001) [6] and study the autocorrelation of absolute (or squared) returns:

$$C_\alpha(\tau) = \text{corr}(|\mathbf{R}_{t+\tau}|^\alpha, |\mathbf{R}_t|^\alpha), \quad \alpha \in \{1, 2\}.$$

Empirically, this correlation decays slowly according to a power law,

$$C_\alpha(\tau) \sim \frac{A}{\tau^\beta}, \quad \beta \in [0.2, 0.4],$$

indicating persistent volatility fluctuations (long memory). For evaluation, we estimate the decay exponent by a log-log linear fit over lags $\tau \in \{1, \dots, \tau_{\max}\}$ where $C_\alpha(\tau) > 0$:

$$\hat{\beta} = -\text{slope}(\log C_\alpha(\tau) \text{ vs. } \log \tau).$$

Our metric is the absolute difference between decay exponents estimated on real and synthetic data:

$$\text{LMSD} = |\hat{\beta}_{\text{real}} - \hat{\beta}_{\text{synth}}|.$$

Unless otherwise stated, we set $\alpha = 1$ (absolute returns) and report $\hat{\beta}$ together with the goodness-of-fit of the linear regression.

6 Utility Evaluation of Downstream Tasks

Most market generator research focuses on statistical similarity while overlooking practical downstream effects [12, 20]. While some work explores option hedging [4, 21], benchmarks like TSGBench lack broader utility experiments. We introduce three downstream tasks **[C2]** under a train-synthetic-test-real (TSTR) framework [14, 20] — option delta hedging (options market maker), portfolio hedging (equity market maker), and alpha generation (hedge fund manager) — demonstrating synthetic data utility in realistic settings [12].

6.1 Overview of Utility Evaluation

All three tasks follow a common protocol. Each task $\mathcal{T}$ is specified by a loss function $\mathcal{L}_{\mathcal{T}}$ over the model's policy θ , and the optimal policy is

$$\theta^* := \arg \min_{\theta} \mathcal{L}_{\mathcal{T}}(\theta), \quad (21)$$

(or $\arg \max$ for reward-type objectives). Policies are trained under the TSTR framework of §6.1.1 on the data partition of §4.1 with rolling-window validation for hyperparameter tuning [14], and scored with the metric toolbox of §6.1.3.

6.1.1 Utility Training and Evaluation Framework. Consider a market generator G_g with synthetic data $\hat{\mathcal{D}}_g$. Training an algorithm A (e.g. a 5-layer perceptron) for task $\mathcal{T}$ on $\mathcal{D}^{\text{train}}$ and $\hat{\mathcal{D}}_g$ yields models M_r and $\hat{M}_g$, respectively. Scoring both on the real test set $\mathcal{D}^{\text{test}}$ by a score S^5 gives s_r and s_g ; the generator is useful for the pair $(\mathcal{T}, A)$ if $s_g > s_r$, and the best generator is G_{g^*} with $g^* := \max\{s_g : g \in \mathcal{G}\}$.

6.1.2 Augmented Training. We additionally train on the augmented set $\hat{\mathcal{D}}_j := \mathcal{D}^{\text{train}} \cup \hat{\mathcal{D}}_j$, yielding $\tilde{M}_j$, to assess whether synthetic data adds value on top of real training data.

6.1.3 Utility Evaluation Metrics. To make downstream evaluations comparable across tasks, we define a common toolbox of utility metrics used by researchers and practitioners. Given a downstream model M for task $\mathcal{T}$, let $\Pi_M := (\pi_{t,i})_{t,i} \in \mathbb{R}^{L \times I}$ denote the trading portfolio generated by M , where $\pi_{t,i}$ is the trading order (position) in asset i at time t . A utility metric U maps the portfolio to a scalar score $u := U(\Pi_M)$, instantiating the TSTR score S of §6.1.1, i.e. $s \equiv u = U(\Pi_M)$ for $M \in \{M_r, \hat{M}_j, \tilde{M}_j\}$. The per-period profit and loss (PnL) of the portfolio is

$$\text{PnL}_t(\Pi_M) := \sum_{i=1}^I \pi_{t,i} r_{t+1,i}, \quad (22)$$

where $r_{t+1,i}$ is the simple return of asset i over $[t, t+1)$. The toolbox evaluates the following metrics over the per-period PnL sequence $\{\text{PnL}_t(\Pi_M)\}_{t=0}^{L-1}$:

[U1] PnL. Total profit and loss over the window, $\text{PnL}(\Pi_M) := \sum_{t=0}^{L-1} \text{PnL}_t(\Pi_M)$.

[U2] Annualized Sharpe ratio. The mean per-period PnL divided by its standard deviation,

$$\text{Sharpe}(\Pi_M) := \frac{\hat{\mu}}{\hat{\sigma}}, \quad (23)$$

$$\hat{\mu} := \frac{1}{L} \sum_{t=0}^{L-1} \text{PnL}_t, \quad \hat{\sigma}^2 := \frac{1}{L-1} \sum_{t=0}^{L-1} (\text{PnL}_t - \hat{\mu})^2,$$

Since the evaluation horizon is a single trading year, no annualization factor (i.e. $\sqrt{L}$) is applied.

[U3] CVaR. The conditional value at risk at level α , i.e. the expected loss in the worst α -tail of the per-period PnL distribution [7],

$$\text{CVaR}_\alpha(\Pi_M) := -\mathbb{E}[\text{PnL}_t \mid \text{PnL}_t \leq \text{VaR}_\alpha], \quad (24)$$

where VaR_α is the lower α -quantile of the per-period PnL distribution.

⁵WLOG, we assume a score on $\mathbb{R}$ where higher is better

[U4] **Win rate.** The fraction of periods with positive PnL,

$$\text{WR}(\Pi_M) := \frac{1}{L} \sum_{t=0}^{L-1} \mathbb{I}\{\text{PnL}_t > 0\}, \quad (25)$$

where $\mathbb{I}\{\cdot\}$ is the indicator function.

[U5] **Max drawdown.** The largest peak-to-trough decline of the cumulative PnL curve $C_t := \sum_{s=0}^t \text{PnL}_s$,

$$\text{MDD}(\Pi_M) := \max_{0 \leq u \leq v \leq L} (C_u - C_v). \quad (26)$$

Let $u_n := U(\Pi_M^{(n)})$ be the value of a metric U on the n -th test window. We report each metric as

$$\bar{u} := \frac{1}{N} \sum_{n=1}^N u_n, \quad \hat{\sigma}_u := \sqrt{\frac{1}{N-1} \sum_{n=1}^N (u_n - \bar{u})^2}, \quad (27)$$

the mean and standard deviation of U across all $N := |\mathcal{D}^{\text{test}}|$ test windows.

Every task below instantiates the same protocol: train $\{M_r, \hat{M}_j, \tilde{M}_j\}$ on $\{\mathcal{D}^{\text{train}}, \hat{\mathcal{D}}_j, \tilde{\mathcal{D}}_j\}$ per §6.1.1–6.1.2, evaluate on $\mathcal{D}^{\text{test}}$, and report the toolbox metrics above.

6.2 Option Delta Hedging

Option hedging is a popular downstream utility benchmark in market generator research [4, 21]. We take the perspective of an options market maker selling a European call option on a cash equity to minimize counterparty risk when the call expires [7].

6.2.1 Problem Definition. Consider a European call at strike K , maturing at time L , on cash equity i with payoff $g(L) := (S_{L,i} - K)_+$, rebalanced at the discrete times $\mathbb{T} := \{t \in \mathbb{N} : 0 \leq t \leq L-1\}$ (end-of-day). The hedger's policy is the hedge position $\Delta := (\Delta_t)_{t=0}^{L-1} \in \mathbb{R}^L$, i.e. the trading portfolio $\Pi_M \equiv \Delta$. The loss is the quadratic replication error of the short call,

$$\mathcal{L}_\Delta(\Delta) := \mathbb{E} \left[\left(g'(L) - \sum_{t=0}^{L-1} \Delta_t \tilde{g}(t) (e^{R_{t+1,i}} - 1) \right)^2 \right], \quad (28)$$

where $\tilde{g}(t) := S_{t,i}/K$ is the moneyness of the contract, $\tilde{g}(L) := (S_{L,i}/K - 1)_+$ its payoff in moneyness units, and $g'(L) := \tilde{g}(L) - c_0/K$ with c_0 the call premium from the Black-Scholes-Merton equation. The optimal policy minimizes this loss,

$$\Delta^* := \arg \min_{\Delta} \mathcal{L}_\Delta(\Delta). \quad (29)$$

6.2.2 Framework. We use a long-short-term-memory (LSTM) network under the loss of equation 28 [7]. While related work feeds raw price data [4, 7, 21], our framework recurrently feeds the moneyness $\tilde{g}(t)$ instead of the raw price, allowing the network to generalize the payoff across strikes. At each step the LSTM takes the triplet $(\Delta_{t-1}, \tilde{g}(t), \tau_t)$ — the previous hedge position, the current moneyness, and the normalized time-to-maturity $\tau_t := 1 - t/L$.

6.2.3 Training. Since the LSTM hedges one asset at a time, each training window $\mathbf{w} \in \mathcal{D}$ is treated as one channel. For each window we append an initial hedge $\Delta(0) = 0$ and an initial moneyness $\tilde{g}(0)$

⁶, drawn from 0.7 to 1.3 at 0.1 increments ⁷, and propagate the log-moneyness recursively as $\log \tilde{g}(t) = \log \tilde{g}(t-1) + R_{t,i}$ ⁸. Models are trained per §6.1.1.

6.2.4 Evaluation. Models $\{M_r, \hat{M}_j, \tilde{M}_j\}$ are scored on $\mathcal{D}^{\text{test}}$ with the toolbox of §6.1.3; we additionally report the hedging loss $\mathcal{L}_\Delta$ of equation 28, aggregated per equation 27.

6.3 Portfolio Hedging

Portfolio hedging mitigates the risk of adverse price movements in a market maker's inventory. Classical market-making models assume dealers hedge inventory risk externally once it becomes sufficiently large [10]; we instantiate this external hedge with $J = 5$ liquid U.S. ETFs (table 1) serving as proxies for market and sector exposures.

6.3.1 Problem Definition. For a time horizon L and a universe of $I = 20$ inventory stocks, let $P_t := (p_{t,i})_{i=1}^I \in \mathbb{R}^I$ be the market maker's inventory at time t , modeled as continuous exposures. The hedge policy $\mathbf{H} := (h_{t,j})_{t,j} \in \mathbb{R}^{L \times J}$ — the trading portfolio $\Pi_M := \mathbf{H}$ — nets the inventory notional exposure against the ETF hedge. The loss is the cumulative absolute net exposure,

$$\mathcal{L}_H(\mathbf{H}) := \sum_{t=1}^{L-1} \left| \sum_{i=1}^I R_{t+1,i} p_{t,i} - \sum_{j=1}^J R_{t+1,j} h_{t,j} \right|, \quad (30)$$

where $R_{t,i}$ and $R_{t,j}$ are the daily log returns of the inventory stocks and ETFs, respectively. The optimal hedge minimizes this loss,

$$\mathbf{H}^* := \arg \min_{\mathbf{H}} \mathcal{L}_H(\mathbf{H}). \quad (31)$$

6.3.2 Inventory Simulation. Since proprietary inventory records are unavailable, a natural alternative is to simulate inventory with dedicated synthetic inventory generators [10]; however, this introduces external uncertainty that obscures the generator's utility in a controlled setting. We therefore evaluate on a static portfolio: $P_t = \mathbf{1}$, $t \in [L]$, the I -dimensional unit vector.

6.3.3 Framework and Training. We apply the same LSTM framework as §6.2, feeding at each step the previous hedge $\mathbf{H}_{t-1}$ and the current returns $R_{t,i}, R_{t,j}$ for all $i \in [I], j \in [J]$. Training follows §6.1.1.

6.3.4 Evaluation. Models $\{M_r, \hat{M}_j, \tilde{M}_j\}$ are scored on $\mathcal{D}^{\text{test}}$ with the toolbox of §6.1.3, reported as the mean and standard deviation across all test windows.

6.4 Alpha Generation

Alpha generation takes the perspective of a hedge fund manager or proprietary trader seeking to capture excess returns by trading the full asset universe. Unlike the hedging tasks, which neutralize an existing exposure, the model takes directional positions to maximize risk-adjusted PnL.

⁶Equivalent to fixing the strike $K = S_{0,i}/\tilde{g}(0)$ for the contract of maturity L

⁷These values correspond to option contracts 10%–30% out-of- and in-the-money relative to the initial price

⁸For a training set of size $N = |\mathcal{D}|$, this yields $N \times 7$ training examples

6.4.1 Problem Definition. For a horizon L and the full universe of $I = 25$ assets (20 stocks and 5 ETFs, table 1), the model M produces the trading portfolio $\Pi_M := (\pi_{t,i})_{t,i} \in \mathbb{R}^{L \times I}$, where $\pi_{t,i}$ is the position in asset i held over $[t, t + 1)$, with per-period PnL as in equation 22. The loss is the negative differentiable Sharpe ratio,

$$\mathcal{L}_\Pi(\Pi_M) := -\text{Sharpe}(\Pi_M) = -\frac{\hat{\mu}}{\hat{\sigma}}, \quad (32)$$

where $\text{Sharpe}(\cdot)$ is defined in equation 23 (and equals the annualized value at the one-trading-year horizon). The optimal portfolio is

$$\Pi^* := \arg \min_{\Pi} \mathcal{L}_\Pi(\Pi) = \arg \max_{\Pi} \text{Sharpe}(\Pi). \quad (33)$$

6.4.2 Framework. We employ the same LSTM architecture as §6.2, with an output head emitting cross-sectional trading orders for all I assets. At each step t , the LSTM feeds the previous order vector Π_{t-1} and the current log returns R_t , and outputs the updated orders $\Pi_t := (\pi_{t,i})_{i=1}^I \in \mathbb{R}^I$. Training follows §6.1.1.

6.4.3 Evaluation. Models $\{M_r, \hat{M}_j, \tilde{M}_j\}$ are scored on $\mathcal{D}^{\text{test}}$ with the toolbox of §6.1.3 reported as the mean and standard deviation across all N test windows.

References

- [1] Yihao Ang, Qiang Huang, Yifan Bao, Anthony K. H. Tung, and Zhiyong Huang. 2023. TSGBench: Time Series Generation Benchmark. *Proc. VLDB Endow.* 17, 3 (2023), 305–318.
- [2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein Generative Adversarial Networks. *Proceedings of the 34th International Conference on Machine Learning* 70 (2017), 214–223.
- [3] Samuel Assefa, Danial Dervovic, Mahmoud Mahfouz, Tucker Balch, Prashant Reddy, and Manuela Veloso. 2019. Generating synthetic data in finance: opportunities, challenges and pitfalls. In *NeurIPS 2019 Workshop on AI in Financial Services: Data, Fairness, Explainability, Trustworthiness, and Privacy*. Vancouver, Canada. <https://ssrn.com/abstract=3634235>
- [4] Nicolas Boursin, Carl Remlinger, Joseph Mikael, and Carol Anne Hargreaves. 2022. Deep Generators on Commodity Markets; application to Deep Hedging. arXiv:2205.13942 [q-fin.RM] <https://arxiv.org/abs/2205.13942>
- [5] Junyoung Chung, Kyle Kastner, Laurent Dinh, Kratarth Goel, Aaron C. Courville, and Yoshua Bengio. 2015. A Recurrent Latent Variable Model for Sequential Data. In *Advances in Neural Information Processing Systems*, Vol. 28. <https://arxiv.org/abs/1506.02216>
- [6] R. Cont. 2001. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance* 1, 2 (2001), 223–236. arXiv:<https://doi.org/10.1080/10.1080/713665670> doi:10.1080/713665670
- [7] Simon Fecamp, Joseph Mikael, and Xavier Warin. 2021. Deep learning for discrete-time hedging in incomplete markets. *The Journal of Computational Finance* (01 2021). doi:10.21314/JCF.2021.006
- [8] Marco Fraccaro, Simon Kamronn, Ulrich Paquet, and Ole Winther. 2017. A Disentangled Recognition and Nonlinear Dynamics Model for Unsupervised Learning. In *Advances in Neural Information Processing Systems*, Vol. 30. <https://arxiv.org/abs/1710.05729>
- [9] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Networks. arXiv:1406.2661 [stat.ML] <https://arxiv.org/abs/1406.2661>
- [10] Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez Tapia. 2011. Dealing with the Inventory Risk. A solution to the market making problem. arXiv:1105.3115 [q-fin.TR] <https://arxiv.org/abs/1105.3115>
- [11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 574, 12 pages.
- [12] Blanka Horvath, Jonathan Plen, and Milena Vulić. 2025. Market Generators: A Paradigm Shift in Financial Modeling. In *Signature Methods in Finance*. Springer Nature Switzerland, Cham, 127–159. doi:10.1007/978-3-031-97239-3_4
- [13] Hans R. Künsch. 1989. The Jackknife and the Bootstrap for General Stationary Observations. *The Annals of Statistics* 17, 3 (1989), 1217–1241. doi:10.1214/aos/1176347265
- [14] Mark Leznik, Patrick Michalsky, Peter Willis, Benjamin Schanzel, Per-Olov Östberg, and Jörg Domaschka. 2021. Multivariate Time Series Synthesis Using Generative Adversarial Networks. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering* (Virtual Event, France) (ICPE '21). Association for Computing Machinery, New York, NY, USA, 43–50. doi:10.1145/3427921.3450257
- [15] Hao Lou, Tianhong Li, and Hao Ni. 2023. PCF-GAN: Generating Sequential Data via the Characteristic Function of Measures on the Path Space. arXiv:2306.07235 [cs.LG] <https://arxiv.org/abs/2306.07235>
- [16] Hao Ni, Lukasz Szpruch, Magnus Wiese, Shujian Liao, and Baoren Xiao. 2021. Conditional Sig-Wasserstein GANs for Time Series Generation. arXiv:2006.05421 [stat.ML] <https://arxiv.org/abs/2006.05421>
- [17] Dimitris N. Politis and Joseph P. Romano. 1994. The Stationary Bootstrap. *J. Amer. Statist. Assoc.* 89, 428 (1994), 1303–1313. doi:10.1080/01621459.1994.10476870
- [18] Vamsi K. Potluru, Daniel Borrajo, Andrea Coletta, Niccolò Dalmaso, Yousef El-Laham, Elizabeth Fons, Mohsen Ghassemi, Sriram Gopalakrishnan, Vikesh Gosai, Eleonora Kreacic, Ganapathy Mani, Saheed Obibatayo, Deepak Paramanand, Natraj Raman, Mikhail Solonin, Srikanth Sood, Svitlana Vyetrenko, Haibei Zhu, Manuela Veloso, and Tucker Balch. 2024. Synthetic Data Applications in Finance. arXiv:2401.00081 [cs.LG] <https://arxiv.org/abs/2401.00081>
- [19] Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollmer. 2021. Autoregressive Denoising Diffusion Models for Multivariate Probabilistic Time Series Forecasting. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 8857–8868. <https://arxiv.org/abs/2101.12072>
- [20] Michael Stenger, Robert Leppich, Ian Foster, Samuel Kounev, and André Bauer. 2024. Evaluation is key: a survey on evaluation measures for synthetic time series. *J Big Data* 11, 66 (May 2024). doi:10.1186/s40537-024-00924-7
- [21] Yuki Tanaka, Ryuji Hashimoto, Takehiro Takayanagi, Zhe Piao, Yuri Murayama, and Kiyoshi Izumi. 2025. CoFinDiff: Controllable Financial Diffusion Model for Time Series Generation. arXiv:2503.04164 [q-fin.CP] <https://arxiv.org/abs/2503.04164>
- [22] Zhengwei Wang, Qi She, and Tomas E. Ward. 2020. Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy. arXiv:1906.01529 [cs.LG] <https://arxiv.org/abs/1906.01529>
- [23] Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. 2020. Quant GANs: Deep Generation of Financial Time Series. *Quantitative Finance* 20, 9 (April 2020), 1419–1440. doi:10.1080/14697688.2020.1730426
- [24] Xinyu Yuan and Yan Qiao. 2024. Diffusion-TS: Interpretable Diffusion for General Time Series Generation. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2403.01742>